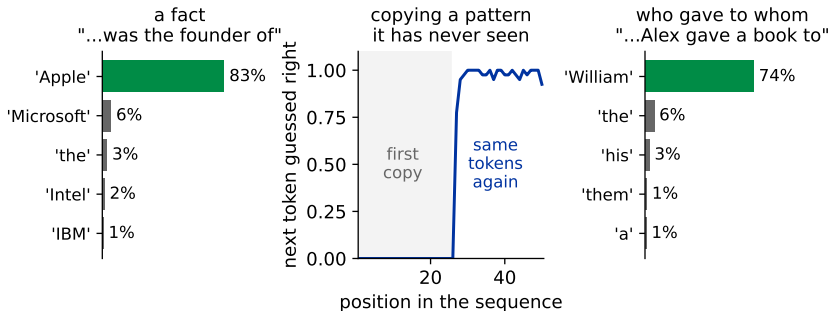


Looking Inside

What mechanistic interpretability is, and why we do it

Three things a small model can do



GPT-2 small (124 million parameters, 2019). Left: "Steve Jobs was the founder of". Middle: 25 random tokens, then the same 25 again - it guesses **0%** of the first copy and **93.5%** of the second.

By the end of this chapter you will know how each of these works **inside** the model - which parts do it, and how we know.

Twenty lectures of black boxes

In chapter 5 we learned to explain a model without ever looking inside it.

- ▶ SHAP splits the prediction between the input features.
- ▶ Permutation importance shuffles a column and watches the score fall.
- ▶ Partial dependence sweeps one feature and plots the response.

Every one of those methods treats the model as a **function you can only query**. Feed it inputs, read the outputs, infer what mattered. None of them ever reads a weight.

That was not a limitation of the methods. It was a decision - and for a gradient-boosted tree it is still the right one. In this chapter we make the other decision.

Predict first

We have complete access to GPT-2 small. Every one of its 124 million numbers is sitting on the disk in front of us, and we can read all of them.

Is knowing every weight the same as understanding the model?

Predict first

We have complete access to GPT-2 small. Every one of its 124 million numbers is sitting on the disk in front of us, and we can read all of them.

Is knowing every weight the same as understanding the model?

No - and the reason is worth stating precisely.

We have the weights the way you have a wiring diagram of a city's power grid: complete, correct, and no help at all in answering "*why did that street go dark?*" **Access is not understanding.** The gap between them is the entire field.

What mechanistic interpretability is

Using a model's **internals** - its weights and activations - to understand what it is doing.

Note what that definition does **not** say. It does not promise a complete reverse-engineering of the network, and the field has become deliberately pragmatic about this: the goal is useful understanding, not a full circuit diagram of a trillion-parameter model.

The through-line of this chapter: it is easy to form a story about what is happening inside a network, and hard to show that the story is true. Most of what we teach is how to tell those two apart.

Outline

Why look inside?

Where it sits on the map

A network small enough to read

Features, circuits, universality

How the field works

A short history

The plan

Why look inside?

Four reasons, each with a case you can point to

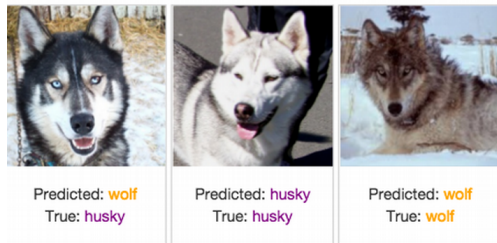
Reason 1 - debugging: right answer, wrong reason

You met this in chapter 5. A classifier separates huskies from wolves, and LIME shows what it looks at: **the snow**.

A black-box method got us that far, and it is a real result. It stops there:

- ▶ *where* in the network does “snow” live?
- ▶ can we remove it without breaking the rest?
- ▶ what else was quietly relying on it?

Those are questions about the **mechanism**, and only the inside can answer them.



Husky vs. wolf: LIME highlights snow pixels. LMU
IML (CC BY 4.0) / Ribeiro et al. (2016).

Reason 2 - trust: does the explanation match the computation?

Every chat model you use can print its reasoning step by step. The natural assumption is that the text is a **transcript** of what the model computed.

That assumption is testable, and it is sometimes false. In the features lecture we look at a traced case where the model's written reasoning describes a calculation the model did **not** do.

The stakes grow with what the model is allowed to do. In the agents lecture a model ran tools on its own; an agent whose reasoning you cannot check is an agent you are trusting on its word.

Lindsey et al. (2025), *On the Biology of a Large Language Model*.

Reason 3 - science: what did it learn that nobody taught it?

A board it was never shown. A small transformer trained only on lists of legal Othello moves - never a picture of the board - turns out to keep an internal copy of the board anyway. You can read the state of every square out of its activations. (Li et al., 2022)

A plan it never wrote down. Asked to write a rhyming couplet, a large model picks the rhyme word *before* it starts writing the second line, and then writes a line that lands on it. (Lindsey et al., 2025)

Neither fact is visible in the output. Both were found by looking inside. The model learned things its training data only implied, and the only way to find out what is to look.

Reason 4 - control: if you can find it, you can turn it

In 2024 Anthropic found a direction inside Claude that switched on for the Golden Gate Bridge - in text, in photos, in any language.

Then they **held it switched on** while the model talked. The result, “Golden Gate Claude”, brought every conversation around to the bridge, and when asked what it was, answered that it *was* the bridge.

It is a joke, and it is also evidence: changing one internal direction changed the model's behaviour in exactly the predicted way. Finding the part and being able to turn it are the same skill.

Templeton et al. (2024), *Scaling Monosemanticity*. We return to this - and to its limits - in the features lecture.

What this does not replace

	Black box (chapter 5)	White box (this chapter)
Works on	any model - boosted trees, a support vector machine, an API you can only call	neural networks whose weights you have
Needs	predictions	weights, activations, and a framework to hook them
Answers	which input moved the output	what the model did
Cost	seconds	the circuit in the causal-methods lecture took a research team months

Most of you will deploy gradient boosting. For that model SHAP is not the weaker tool - it is the **only** tool. This chapter is not an upgrade over chapter 5. It is a different instrument, for a different object.

Where it sits on the map

Chapter 5 drew a map of interpretability. This chapter adds a region.

The axes you already know

Chapter 5 sorted every method along a few questions:

Question	One answer	The other
Is the model readable by design?	yes - a glass box (a short tree, a linear model)	no - explain it afterwards (<i>post hoc</i>)
Does the method care which model?	no - <i>model-agnostic</i> (SHAP, PDP)	yes - <i>model-specific</i>
What does it explain?	<i>one</i> prediction (local)	the <i>whole</i> model (global)

Mechanistic interpretability is **post hoc** and **model-specific**, and it works at both scales: one prompt, or a feature across all prompts.

What is genuinely new is a fourth question: **what is the explanation made of?** Chapter 5's answers are made of *inputs*. This chapter's are made of *parts of the model*.

The map

	asks the model questions	reads its gradients	reads its weights and activations
the whole model	permutation importance PDP / ALE SHAP summary	(rare)	circuits probes sparse autoencoder features linear-model coefficients*
one prediction	LIME SHAP (one row) counterfactuals	saliency maps integrated gradients Grad-CAM	logit lens activation patching attribution graphs
	chapter 5	chapter 6 + the vision lecture	this chapter

*You have done mechanistic interpretability before: reading a linear model's coefficients *is* reading its weights. It is easy there because the model has one layer. This chapter is about what that habit costs on a model with twelve.

Why not just use a glass box?

Chapter 5's first lecture made the case for models that are readable by design: a shallow tree, a sparse linear model, a rule list. When one of those is accurate enough, use it.

For this chapter's models it is not an option:

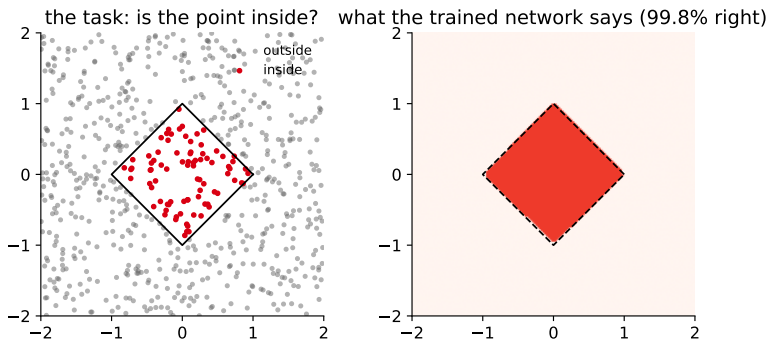
- ▶ there is no glass-box model that writes a paragraph, translates, or answers questions;
- ▶ the models people actually deploy for language and vision are the ones we understand least.

So the choice is not “interpretable model or accurate model”. The model is already chosen, it is a neural network, and the only question left is how much of it we can read.

A network small enough to read

Before GPT-2: the whole method, on a network with four neurons

The task



Is the point (x, y) inside the diamond $|x| + |y| \leq 1$? A network with **2 inputs, 4 hidden ReLU units, 1 output**, trained here on a laptop in under a minute. It gets **99.8%** of the plane right.

Predict first

Four hidden units. A diamond has four edges.

What do you think each hidden unit does?

Predict first

Four hidden units. A diamond has four edges.

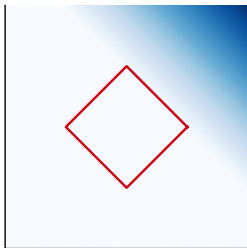
What do you think each hidden unit does?

Recall what one ReLU unit computes: $h = \max(0, w_1x + w_2y + b)$. It is **zero** on one side of the line $w_1x + w_2y + b = 0$ and grows linearly on the other.

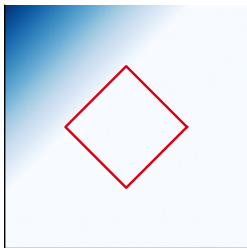
So each unit is a **line** with an “on” side. Four units, four lines. Where would you put them?

Each unit, over the whole plane

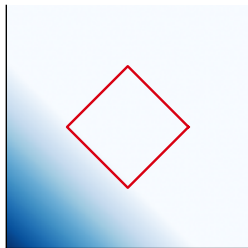
unit 1: fires outside
the top-right edge



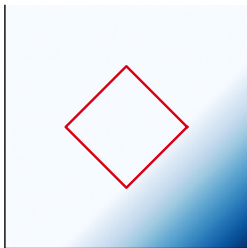
unit 2: fires outside
the top-left edge



unit 3: fires outside
the bottom-left edge



unit 4: fires outside
the bottom-right edge



Darker blue means the unit fires harder. Each unit is **silent inside the diamond** and switches on only once you cross one particular edge.

We just named all four neurons: “outside the top-right edge”, “outside the top-left edge”, and so on. Each one means one thing, and we can say what.

Reading it off the weights

The same names, without drawing anything - just the numbers the network learned:

	w_1	w_2	b	fires when	output weight
unit 1	10.13	10.17	-9.59	$x + y > 0.94$	-9.24
unit 2	-8.62	8.44	-7.97	$-x + y > 0.93$	-10.44
unit 3	-9.64	-9.77	-8.87	$-x - y > 0.91$	-7.12
unit 4	8.75	-8.81	-8.12	$x - y > 0.92$	-9.06

$$\text{output} = 6.07 - 9.24 h_1 - 10.44 h_2 - 7.12 h_3 - 9.06 h_4 \quad (\text{inside if } > 0)$$

Each “fires when” column is the unit’s own inequality divided by its weight - e.g.

$10.13x + 10.17y > 9.59 \Rightarrow x + y \gtrsim 0.94$, the diamond’s edge $x + y = 1$ moved slightly inward.

Two points, by hand

(0.5, 0.25) - **inside**

unit 1:

$$10.13(0.5) + 10.17(0.25) - 9.59 = -1.98$$

$$\text{units 2, 3, 4: } -10.17, -16.13, -5.95$$

all negative, so after ReLU: $h = (0, 0, 0, 0)$

$$\text{output} = 6.07 - 0 = +\mathbf{6.07} \quad \text{inside}$$

(1, 0.5) - **outside**

$$\text{unit 1: } 10.13(1) + 10.17(0.5) - 9.59 = +5.63$$

$$\text{units 2, 3, 4: } -12.37, -23.39, -3.77$$

$$\text{after ReLU: } h = (5.63, 0, 0, 0)$$

$$\text{output} = 6.07 - 9.24 \times 5.63 = -\mathbf{45.9}$$

outside

Inside the diamond every unit is silent, and the output is just the bias, +6.07. Step past an edge and that edge's unit fires, and its large negative weight drags the output below zero.

The circuit, in one sentence

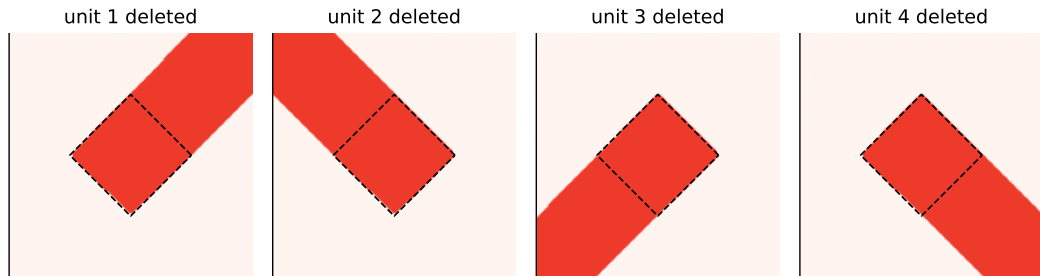
“If you are past any edge, you are out.”

That is the whole algorithm, and we got it without guessing: four **features** (one edge each), combined by the output unit into a **circuit** (an AND of four “not past this edge” conditions).

Those two words - **feature** and **circuit** - are the whole vocabulary of this chapter. A feature is something the network represents. A circuit is features wired together to compute something.

But so far we have only *looked*. The reading could still be wrong.

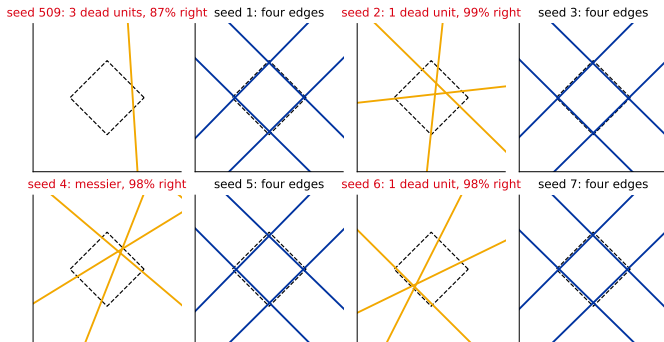
Now test it



Delete one unit (set it to zero). Any one deletion opens up **exactly its edge**: about **15.5%** of the plane becomes “inside”, and only **0.1%** of the true diamond is lost.

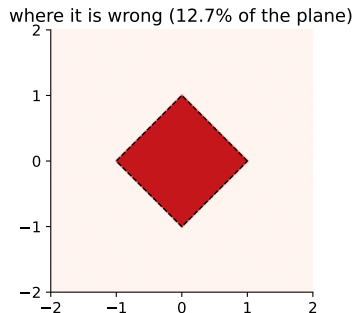
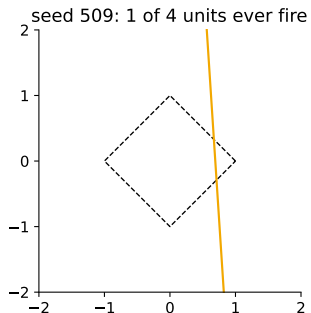
The reading **predicted** this before we ran it. That is the difference between a story and evidence: **looking is a hypothesis, intervening is the test.**

Train it again



Eight training runs, same data, different random start; lines are the live units. **4 of 8** runs find the four-edge network - the same four edges each time, in a different order, so “unit 1” is a different edge in each run. The other four got stuck somewhere messier.

The stuck network is readable too



Seed 509: three of its four units **never fire** anywhere in the plane - dead ReLUs, the failure from the lecture on initialization. The network answers “outside” for every point.

Its accuracy is **87.3%** - exactly the share of the plane that is outside the diamond. The accuracy said “pretty good”. The weights said “learned nothing”, in one glance.

Why this does not scale

	our toy	GPT-2 small
hidden units	4	36,864 MLP neurons, plus 144 attention heads
inputs per unit	2 - you can plot them	768 - you cannot
does each unit mean one thing?	yes	mostly no
can you check a reading?	delete a unit, look	delete a unit - and the model may compensate

Every hard problem in this field is in that table. We cannot draw the units, so we need other ways to read them. Units mean several things at once, so the neuron may be the wrong thing to read. And deleting a part does not always reveal what it did. Each of those gets a lecture.

Features, circuits, universality

The three claims the field is built on

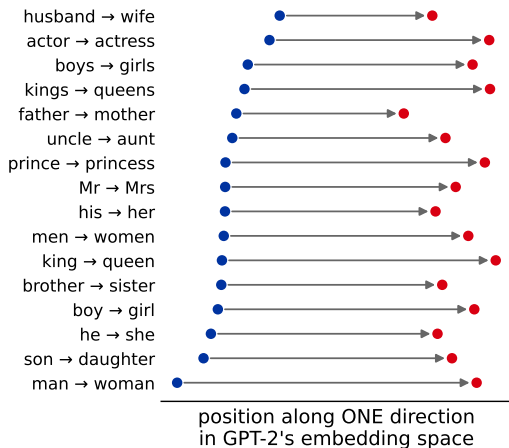
Three claims

In 2020 a group at OpenAI wrote down the working assumptions of the field as three claims - deliberately labelled *speculative*:

1. **Features.** Networks represent meaningful things, and those representations can be found and understood.
Our toy: four edges.
2. **Circuits.** Features are connected by weights into small, readable algorithms.
Our toy: “past any edge, you are out”.
3. **Universality.** Similar features and circuits show up in different networks trained on similar tasks.
Our toy: 4 of 8 runs found the same four edges.

Olah et al. (2020), *Zoom In: An Introduction to Circuits*.

A feature is a direction



GPT-2 small stores each token as 768 numbers. Take 16 male/female word pairs and average the 16 “female minus male” differences into one direction.

Every pair moves the same way along it.

Not circular: rebuild the direction *without* a pair, and that pair still moves the right way - **16 of 16**. Each pair's difference has cosine **0.60** with the shared direction; a random direction gets **0.03**.

Nobody told the model about gender.
It found a direction for it.

Arithmetic with directions

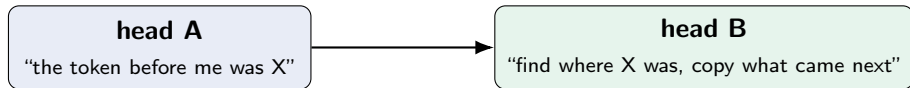
If meanings are directions, you can add and subtract them. Nearest tokens to the result, in GPT-2 small's own embeddings:

sum	nearest token	cosine
king - man + woman	queen	0.71
walked - walk + run	ran	0.74
cats - cat + dog	dogs	0.85
bigger - big + small	smaller	0.80
Paris - France + Italy	Paris (again), then Italy	0.63

The last row is kept on purpose. “Rome” is fifth. Directions are real, but they are not magic, and a slide that only shows the analogies that worked is the first cherry-pick of the chapter.

A circuit: features wired together

In the toy, the circuit was four edge-features feeding one output. In a real language model the first circuit you will meet is the one behind the middle panel of the opening slide - copying a pattern it has never seen:



Neither head does the task alone. The behaviour lives in the **connection**. How it works is in the next core lecture; how to read it straight off the weights is in an optional one.

Olsson et al. (2022), *In-context Learning and Induction Heads*.

Universality

Why should a lecture on GPT-2 small tell you anything about any other model?
Because of the third claim.

- ▶ **Edge and curve detectors** have been reported in many different image networks, of different architectures, trained separately.
- ▶ **The copying circuit** from the last slide turned up in every model size the original study checked, from one-layer toys to large models, and in many models since.
- ▶ Our toy, in miniature: different runs, same four edges.

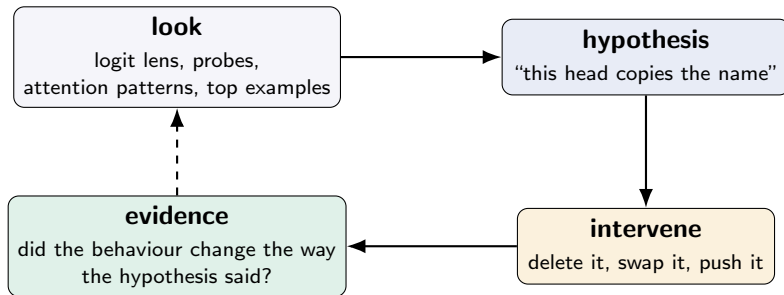
Universality is an **empirical** claim, and only partly true - our toy found the clean solution in half its runs, not all. But it is the reason studying small models is worth anything, and the reason the field bothers to name what it finds.

Olah et al. (2020); Olsson et al. (2022).

How the field works

Look, guess, intervene - and believe only the last one

The loop



The toy network went round this loop once: we *looked* at the units, guessed "each is an edge", *deleted* one, and the edge opened. The rest of the chapter is the same loop on models where every step is harder.

What each kind of evidence proves

What we see	What it proves
a head attends to the right word	nothing yet - it may write nothing
a probe reads the information out	it is present , not that it is used
a neuron's top examples share a theme	maybe - or a cherry-pick
deleting a part changes nothing	not that the part is useless - something may cover for it
copying a part's value into another run moves the answer	the information is there, and it matters

This table is the chapter. Each row gets measured on a real model before we are done, including the fourth, which is the strangest result we will see.

Two analogies, and where they break

Reverse-engineering a program

The weights are a compiled binary: complete and executable, but unreadable. We want the source code - variables (features) and functions (circuits).

Where it breaks: nobody wrote this program, so there may be no clean source code to recover.

Neuroscience, with superpowers

A neuroscientist would give anything to record every neuron at once, change any of them precisely, and run a thousand identical copies of the same brain. We can do all three.

Where it breaks: a brain is one fixed object; we can retrain a network and get a different one.

Both analogies say the same thing: the problem is not **access**, it is **translation** - from numbers into concepts a person can check.

A short history

Thirteen years, from heatmaps to circuits

The field, in dates

	vision models	language models
2013	saliency maps: the gradient as a heatmap	
2015	DeepDream	
2017	feature visualization; integrated gradients	
2020	circuits read out of a vision network	the logit lens
2021		the mathematics of transformer circuits
2022		induction heads; editing a stored fact; superposition; a full circuit for “who gave what to whom”
2023		sparse autoencoders at scale
2024		Golden Gate Claude
2025		attribution graphs
2026		named an MIT Technology Review breakthrough

It began with **images**: gradients as heatmaps, then optimizing an input to see what a neuron wants, then circuits read out of a vision network. From 2020 the same questions moved to **language models**, and most of the chapter lives there.

Where the field is now

In January 2026 MIT Technology Review named mechanistic interpretability one of its ten breakthrough technologies of the year. Money and attention have followed.

The people doing the work are more cautious than the headline. From the 2025 paper behind the most impressive results in this chapter:

“...satisfying insight for about a quarter of the prompts we’ve tried.”

Both are true. The tools now say useful, checkable things about what is inside a language model. They do not yet explain one. Hold both - the chapter will keep showing you each.

Lindsey et al. (2025), *On the Biology of a Large Language Model*.

The plan

Four core lectures, four optional ones

The chapter

lecture		what it adds
1	What we are doing, and why	core this one
2	Seeing what a network sees	optional the same questions on an image model
3	Opening the box	core <i>reading</i> a language model
4	Circuits read off the weights	optional circuits from the weights alone, no data
5	Does it actually do that?	core <i>proving</i> what a part does
6	Where facts live	optional where a fact is stored, and editing it
7	Features	core finding the right <i>units</i> to read
8	Personas and model diffing	optional what fine-tuning changed inside

The four **core** lectures are a complete path on their own. Each **optional** one needs only the core lectures before it, so any of them can be skipped without breaking the rest.

Why a small, old model

The language lectures run on GPT-2 small: **12 layers, 12 heads per layer, 144 heads** in total, and a residual stream of width **768**. It is seven years old.

Why it is the right choice

- ▶ every interpretability tool targets it;
- ▶ it is where the field's best-documented circuit was found;
- ▶ it runs on a laptop, so **every figure in these lectures is one you can reproduce**.

What transfers, and what does not

- ✓ the residual-stream picture;
- ✓ the causal methods, all of them;
- ✓ the standards of evidence;
- ✗ the **specific circuits**. What we find is a fact about GPT-2, not about every model.

The field has largely **retired** toy-model interpretability as a research direction, for the reason in the right-hand column. It is still the only sane place to *learn* it. Training ground, not frontier.

What you will be able to do

After the four core lectures:

- ▶ split a model's output into exact per-component contributions, and say which parts mattered;
- ▶ test a claim about a component by intervening, and choose the right intervention;
- ▶ recognise the four standard ways an interpretability claim fools its author;
- ▶ train a probe on activations - the technique most likely to be useful in a job;
- ▶ open a real open-weight model's features in a browser and read them critically.

What you will **not** be able to do: explain how a frontier model answers a question. Nobody can yet. A lecture that promised it would be lying to you.

Two tools

TransformerLens

2022 · Neel Nanda

github.com/TransformerLensOrg/TransformerLens

A Python library that runs a model and keeps **every** intermediate value instead of throwing it away, with hooks to read or change any of them mid-run. All the language-model figures in this chapter come from it.

Neuronpedia

Johnny Lin · Decode Research

neuronpedia.org

A website for browsing the features of real open models - what each one fires on, what it pushes the output towards. Nothing to install, no GPU. The cheapest way to start.

Recap

- ▶ **Mechanistic interpretability** explains a model with its *parts* - weights and activations - instead of its inputs. Different instrument, not a better one.
- ▶ **Four reasons** to look inside: debugging, trust, science, control.
- ▶ A **four-neuron network** can be read completely: four features (edges), one circuit (“past any edge, you are out”), confirmed by deleting a unit.
- ▶ The vocabulary: **features** (often directions), **circuits**, **universality**.
- ▶ The method: **look, hypothesise, intervene** - and only the last step is evidence.

And the warning from the stuck network: a number that looks good (87% accuracy) can hide a model that learned nothing. Reading the inside is how you find out.

Next: opening the box.

From four neurons to GPT-2 small. What the “residual stream” is and why it makes the whole model a sum, how to watch an answer being built layer by layer, how to ask the model yes/no questions about its own activations - and why every one of those is still only *looking*.

Optional, if it is scheduled first: *Seeing what a network sees* - the same questions on an image model, where you can look at a feature with your own eyes.